JP MORGAN CHASE

Java Developer Interview Guide

19 in-depth topics across Core Java, Spring Boot, Microservices, System Design and Managerial Rounds

---

Designed for: Java developers preparing for mid-level / senior backend interviews
How to use: Learn the concept, explain the internal working, practise the code, then answer the follow-up questions.

Preparation note: Adapt sample project stories and metrics to your actual experience. Never present an illustrative incident as a real production event.

How to answer technical interview questions

For most technical questions, structure your response in three stages: **definition** (what it is), **internal working** (how it works), and **real-world application** (when you use it and what trade-offs it introduces).

- Start with a direct, concise definition.
- Explain the execution flow or internal mechanism.
- Give a realistic banking or payment-system example.
- Mention failure cases, performance implications, and security or data-consistency concerns.
- Finish with the trade-off or reason you selected the approach.

Contents

Round	Topics
1. Core Java and DSA	1 HashMap internals; 2 Longest substring; 3 Linked-list cycle; 4 Multithreading; 5 equals/hashCode
2. Spring Boot and Microservices	6 @Transactional; 7 Self-invocation; 8 Idempotent APIs; 9 Downstream failures; 10 Slow API troubleshooting
3. System Design	11 Highly available payment system; 12 Data consistency; 13 Memory leaks and thread pools; 14 Concurrent reques
4. HR and Managerial	16 Production incident; 17 Why JP Morgan Chase; 18 Technical disagreement; 19 Career goals

Round 1 — Core Java and DSA

1. HashMap internals, collisions and resizing

Definition: HashMap stores key-value pairs and provides average-case O(1) insertion, lookup and deletion when hashes are well distributed.

How put() works

- Java calculates the key's hash code and applies internal hash spreading.
- The hash and current table capacity determine a bucket index.
- If the bucket is empty, a node is inserted.
- If occupied, HashMap checks hashes and key equality. An equal key has its value replaced; otherwise another node is added.
- The map checks whether its resize threshold has been reached.

A simplified bucket index is **(capacity - 1) & hash**. For example, with capacity 16, integer hashes 1 and 17 map to bucket 1 because 1 & 15 = 1 and 17 & 15 = 1.

Collision handling and treeification

A collision occurs when different keys map to the same bucket. HashMap can store multiple entries in one bucket. Modern Java implementations use linked nodes and may convert a heavily populated bucket into a red-black tree when implementation conditions are met. Common thresholds are 8 nodes for treeification and a minimum table capacity of 64; otherwise resizing may be preferred.

Resizing and complexity

The default initial capacity is 16 when the default-constructed map is first populated; the default load factor is 0.75. The threshold is capacity × load factor, so 16 × 0.75 = 12. As the map grows beyond the threshold, capacity normally doubles and entries are redistributed.

Operation	Typical complexity
put / get / remove	Average O(1)
Collision-heavy bucket	Can be O(n); tree bins can improve lookup toward O(log n)

Interview follow-ups: HashMap is not thread-safe; use ConcurrentHashMap or synchronization for shared concurrent mutation. HashMap permits one null key and multiple null values. Equal keys must have equal hash codes. Avoid mutating fields used in a key's equals/hashCode while it is stored in a hash collection.

2. Longest substring without repeating characters

Problem: For a string such as `abcabcbb`, return 3 because a longest valid substring is `abc`.

Approach: Sliding window with two pointers and a map recording each character's most recent index. When a repeated character appears inside the current window, move the left pointer just past its previous position.

```
import java.util.HashMap;
import java.util.Map;

public class LongestSubstring {
    public static int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> lastSeen = new HashMap<>();
        int left = 0, maxLength = 0;

        for (int right = 0; right < s.length(); right++) {
            char current = s.charAt(right);

            if (lastSeen.containsKey(current)) {
                left = Math.max(left, lastSeen.get(current) + 1);
            }
            lastSeen.put(current, right);
            maxLength = Math.max(maxLength, right - left + 1);
        }
        return maxLength;
    }
}
```

Why Math.max? A repeated character may have appeared before the current window. Math.max prevents the left pointer from moving backwards.

Complexity: O(n) time and O(k) space, where k is the number of distinct characters tracked. Java char is a UTF-16 code unit; use code-point-aware logic if the problem defines characters as Unicode code points.

Follow-up: Explain the difference between substring (contiguous) and subsequence (not necessarily contiguous).

3. Cycle detection in a linked list

A cycle exists when a node eventually points back to a previously visited node, so traversal never reaches null. Floyd's tortoise-and-hare algorithm uses two pointers: slow moves one node and fast moves two.

```
public static boolean hasCycle(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Why it works: Once both pointers are in the cycle, the fast pointer gains one node per iteration. Their relative distance eventually becomes zero modulo the cycle length.

Complexity: O(n) time and O(1) extra space. To find the cycle entry, reset one pointer to head after the meeting, then move both one step at a time; their next meeting point is the entry.

4. Multithreading and shared objects

Multiple threads can execute concurrently in one process. In a Spring Boot service, different request threads may access the same singleton service, cache, or mutable object.

```
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
}
```

The expression count++ is not atomic: it reads, increments, and writes. Two threads can read the same old value and overwrite each other's updates. Synchronizing both methods on the same object monitor protects these operations.

- **AtomicInteger:** useful for atomic numeric updates.
- **ReentrantLock:** explicit lock/unlock, timed acquisition and other lock features.
- **volatile:** provides visibility and ordering guarantees, but does not make count++ atomic.
- **Immutability / thread confinement:** reduce shared mutable state.
- **Deadlock:** threads wait on locks held by each other. Use consistent lock ordering and avoid unnecessary nested locks.

The Java Memory Model defines visibility, atomicity and ordering guarantees between threads. In production, inspect thread dumps, blocked threads, lock contention and latency.

5. equals() vs hashCode()

equals() determines logical equality. **hashCode()** returns an integer used by hash-based collections to locate a bucket.

```
import java.util.Objects;

class Employee {
    private final int id;
    private final String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Employee)) return false;
        Employee other = (Employee) obj;
        return id == other.id && Objects.equals(name, other.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }
}
```

- If two objects are equal according to equals(), they must have the same hash code.
- Same hash code does not imply that two objects are equal.
- equals() must be reflexive, symmetric, transitive, consistent, and false for null.
- Overriding equals() without hashCode() can cause logically equal objects to behave incorrectly in HashSet or HashMap.

Round 2 — Spring Boot and Microservices

6. Internal working of @Transactional

@Transactional declaratively manages a database transaction. In a typical proxy-based Spring setup, a proxy intercepts the external method call, asks a transaction manager to begin or join a transaction, invokes the business method, and commits or rolls back based on the outcome.

Bank transfer example

```
@Service
public class TransferService {
    private final AccountRepository repository;

    public TransferService(AccountRepository repository) {
        this.repository = repository;
    }

    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        Account from = repository.findById(fromId).orElseThrow();
        Account to = repository.findById(toId).orElseThrow();

        if (from.getBalance().compareTo(amount) < 0) {
            throw new IllegalArgumentException("Insufficient balance");
        }

        from.debit(amount);
        to.credit(amount);
        repository.save(from);
        repository.save(to);
    }
}
```

This is a simplified example. Real financial flows need appropriate decimal arithmetic, concurrency control, idempotency, auditability and a carefully designed transaction boundary.

Rollback rules

By default, Spring rolls back for unchecked exceptions (RuntimeException and subclasses) and Error. Checked exceptions do not cause rollback by default. Use rollbackFor when checked exceptions must trigger rollback.

```
@Transactional(rollbackFor = Exception.class)
public void processPayment() throws Exception {
    // Database operations
}
```

Propagation

Propagation	Meaning
REQUIRED (default)	Join an existing transaction or create one
REQUIRES_NEW	Suspend current transaction and create an independent one
SUPPORTS	Join if a transaction exists; otherwise run without one
MANDATORY	Require an existing transaction
NOT_SUPPORTED	Suspend existing transaction and run without one
NEVER	Fail if a transaction exists
NESTED	Use nested/savepoint behavior when supported

Isolation levels

Level	General guarantee
READ_UNCOMMITTED	May permit dirty reads, where supported
READ_COMMITTED	Prevents dirty reads
REPEATABLE_READ	Stronger repeat-read guarantees; database behavior differs
SERIALIZABLE	Strongest standard isolation; can reduce concurrency

Self-invocation trap: a method calling another @Transactional method on the same object through this usually bypasses the proxy. Move the transactional method to another Spring bean or use an appropriate weaving approach. Also remember that REQUIRES_NEW may commit independently of an outer transaction.

Interview answer: “Spring typically uses an AOP proxy to manage the transaction boundary. The transaction manager begins or joins the transaction, and commits or rolls back according to the configured rules. I pay attention to propagation, isolation, rollback rules, proxy boundaries and self-invocation.”

7. Self-invocation issues in Spring

Self-invocation occurs when one method directly calls another method on the same object. Proxy-based features such as @Transactional, @Async, @Cacheable and method-level security may not be applied to that internal call.

```
@Service
class ReportWorker {
    @Async
    public void generateAsync() {
        // Expensive report generation
    }
}

@Service
class ReportService {
    private final ReportWorker worker;

    ReportService(ReportWorker worker) {
        this.worker = worker;
    }

    public void generateReport() {
        worker.generateAsync(); // Goes through Spring proxy
    }
}
```

Enable asynchronous processing and configure an appropriate executor. Exceptions from async void methods do not propagate to the original caller like synchronous exceptions; use a suitable result type or async exception handler.

8. Idempotent REST APIs

An operation is idempotent when repeating the same request has the same intended effect as performing it once. This is critical in payments: the server may process a charge but the response may be lost, causing the client to retry.

HTTP method	Generally idempotent?	Example
GET	Yes	Read account details
PUT	Yes	Set resource to specified state
DELETE	Yes	Delete a resource
POST	Not inherently	Create a payment
PATCH	Depends	Partial update

A client can send an Idempotency-Key header with a unique request identifier. The server should atomically register the key, associate it with the caller and request payload, execute the operation safely, persist the outcome, and return

the stored result for a valid retry.

```
CREATE TABLE idempotency_record (
    idempotency_key VARCHAR(100) PRIMARY KEY,
    request_hash VARCHAR(128) NOT NULL,
    status VARCHAR(30) NOT NULL,
    response_body TEXT
);
```

- Use a unique database constraint to protect against concurrent duplicate submissions.
- Reject reuse of the same key with a different payload.
- Track in-progress and completed requests and define retention.
- Coordinate idempotency state and business changes in a transaction where possible.
- Do not infer that a payment failed just because the client timed out.

Idempotency is not merely deduplication by amount and customer: two identical purchases may be legitimate separate requests.

9. Handling downstream failures

A downstream service is a dependency your service calls, such as a bank gateway, customer service, or notification provider. A slow dependency can consume request threads and connections and cause cascading failures.

Pattern	Purpose
Timeout	Stop waiting beyond a defined deadline
Retry with backoff and jitter	Retry transient failures without synchronized retry storms
Circuit breaker	Temporarily stop calls to a repeatedly failing dependency
Bulkhead	Isolate resource pools so one dependency cannot consume all capacity
Fallback / async recovery	Return a safe alternative only if business rules allow it; otherwise persist and reconcile

Resilience4j offers circuit breaker, retry, rate limiter, bulkhead and time limiter modules. Configure timeouts and retry policies based on the end-to-end latency budget and dependency behavior.

Payment warning: a timeout can occur after the bank has processed a charge. Do not blindly retry a non-idempotent payment. Reuse the same idempotency key and reconcile uncertain outcomes.

Interview answer: “I combine bounded timeouts, selective retries, circuit breakers and bulkheads. For critical payment flows, I preserve an uncertain state and reconcile it rather than returning a false failure or issuing a duplicate charge.”

10. Troubleshooting slow APIs

Treat latency as a symptom, not a diagnosis. First determine whether the issue affects one endpoint or the whole service, then use metrics and traces to isolate the slow stage.

- **Database:** execution plans, missing indexes, full scans, lock waits, N+1 queries, connection pool saturation and oversized results.
- **JVM:** garbage collection pauses, heap utilization, allocation rate, CPU hot spots and memory leaks.
- **Threads / network:** active and queued requests, blocked threads, connection pools, downstream latency, DNS and TLS delays.
- **Tools:** Spring Boot Actuator, Micrometer, Prometheus, Grafana, OpenTelemetry, Java Flight Recorder, thread dumps and database execution plans.

Example: fetching 100 orders and then querying each customer's details separately can create 101 database queries. Consider fetch joins, entity graphs, projections or batch fetching.

Interview answer: “I use metrics and distributed traces to identify the slow component, verify the cause with logs or profiling, make a targeted change, and validate the result using p95/p99 latency and error rates.”

Round 3 — System Design

11. Design a highly available payment system

A payment system must handle high traffic, duplicate requests, network failures, partial failures and financial correctness. The key goal is to provide a traceable and correct outcome, not merely a fast response.

Illustrative architecture

Component	Responsibility
Clients / API Gateway	Authentication, routing, request validation and rate limiting
Payment Service	Payment state machine, idempotency and business rules
Risk Service	Fraud and risk checks
Payment database	Durable payment state, constraints and audit records
Bank / payment gateway	External authorization or processing
Transactional outbox + Kafka	Reliable publication of payment events
Notification service	Email, SMS or push updates
Reconciliation service	Compare internal records with provider records

Payment flow

- Client submits payment with an idempotency key.
- Gateway authenticates the client and applies rate limits.
- Payment service validates the request and creates a durable payment record.
- Risk checks and required business validations run.
- Payment service coordinates authorization or processing with the provider.
- Payment state is updated based on confirmed outcomes.
- Events are published for notifications, reporting and reconciliation.
- Reconciliation detects differences between internal and provider records.

Possible states include CREATED, PENDING, AUTHORIZED, CAPTURED, FAILED, CANCELLED, REFUND_PENDING and REFUNDED. A timeout after sending a bank request may require an UNKNOWN or PENDING state until the result is confirmed.

Availability and correctness

- Run multiple service instances across availability zones with health checks and load balancing.
- Use database replication, tested failover, backups and recovery procedures.
- Use stateless application instances where practical.
- Use timeouts, bounded retries, circuit breakers, bulkheads and monitoring.
- Use decimal arithmetic, database constraints and concurrency control for financial operations.
- Consider a double-entry ledger: each financial movement is represented by balanced debit and credit entries.
- Monitor payment outcomes, queue lag, latency, errors and reconciliation discrepancies.

Capacity estimation example

If a service receives 2,000 requests per second and average request duration is 200 ms (0.2 seconds), Little's Law gives an approximate average of 2,000 × 0.2 = 400 requests in flight. This is not a server count. Capacity planning must also account for bursts, tail latency, retries, CPU, memory, database connections and load-test results.

Interview answer: “I design for durable payment state, idempotent requests, controlled concurrency and recovery from uncertain external outcomes. I separate synchronous payment decisions from asynchronous notifications and reporting, and use reconciliation to detect discrepancies.”

12. Microservices data consistency

Each microservice may own its database. This supports independent deployment and scaling but makes cross-service transactions difficult.

Strong consistency vs eventual consistency

Strong consistency provides a view of committed state according to the selected consistency model. Eventual consistency allows dependent systems to temporarily differ and converge when processing succeeds. Use strict financial controls for authoritative ledger state; notifications and some reporting can be asynchronous.

Saga pattern

A saga coordinates a business process through multiple local transactions. Example: create order → reserve inventory → charge payment → confirm order. If a later step fails, compensating actions may release inventory or refund a charge. Compensation is a new business operation, not a database rollback, and can itself fail.

Choreography vs orchestration

- **Choreography:** services react to events and trigger subsequent work. It reduces central coordination but can make the overall flow harder to understand.
- **Orchestration:** a coordinator explicitly directs steps and failure paths. It improves workflow visibility but adds a coordination component.

Transactional outbox

The dual-write problem occurs when a service commits a database update but fails to publish the corresponding Kafka event. The outbox pattern writes the business change and an event record in the same local database transaction. A separate publisher reads and publishes the event. Delivery may happen more than once, so consumers should be idempotent.

Interview answer: “I use local ACID transactions, sagas for multi-service workflows, transactional outbox for reliable event publication, and idempotent consumers. I explicitly define which data needs strong consistency and which can converge asynchronously.”

13. Memory leaks and thread pool exhaustion

Memory leaks

A memory leak occurs when objects are no longer needed but remain reachable, preventing garbage collection.

```
private static final List<byte[]> cache = new ArrayList<>();

public static void storeData() {
    cache.add(new byte[1024 * 1024]); // Retained indefinitely
}
```

- Static collections that grow without bounds.
- Caches without eviction policies.
- Listeners or callbacks that are never removed.
- ThreadLocal values not cleaned up in long-lived worker threads.

- Unbounded queues or buffers and retained references.

Investigate heap usage and GC metrics, capture heap dumps, compare object counts and retained heap, inspect dominator trees and paths to GC roots, then verify that memory stabilizes after the fix.

Thread pool exhaustion

When all worker threads are busy and incoming work keeps arriving, requests wait or are rejected. An unbounded queue can grow without limit.

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    10, 20, 60, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(200),
    new ThreadPoolExecutor.AbortPolicy()
);
```

This is an illustrative bounded executor. Choose pool size, queue capacity and rejection policy based on workload, CPU, downstream limits and latency objectives.

Symptom	Possible cause / investigation
Heap continually grows	Leak or legitimate growth; inspect heap dumps and GC
Workers blocked	Slow dependency or lock contention; inspect thread dump and traces
Queue grows	Arrival rate exceeds processing capacity; inspect queue depth and throughput
Frequent GC	Allocation pressure or insufficient effective heap; inspect GC logs
Rejected tasks	Executor or queue saturation; inspect executor metrics

Increasing thread count is not always a fix. If all threads wait on the same slow database, more threads can worsen database overload.

14. Handling thousands of concurrent requests

Concurrency is the number of requests in progress; throughput is the number completed per unit time. They are related but not interchangeable.

- **Stateless instances:** keep request-specific mutable state out of singleton fields; store durable state in suitable shared systems.
- **Connection pools:** use controlled database connection limits to protect the database.
- **Backpressure:** queue, slow, reject or shed work when downstream capacity is reached.
- **Caching:** cache frequent reads only when freshness and invalidation rules allow it.
- **Async processing:** move noncritical notifications and reporting to background workers.
- **Rate limiting:** limit excessive traffic by user, key, account or another meaningful identity.
- **Load balancing and horizontal scaling:** add application instances while checking shared database and dependency bottlenecks.

Traditional servlet stacks commonly use request-processing threads that can block on I/O. Reactive I/O or Java virtual threads can help with many I/O-bound workloads, but neither removes CPU, memory, database connection or downstream capacity limits.

Interview answer: “I define throughput, p95/p99 latency and error targets, load-test realistic traffic, identify the first saturated component, then scale or optimize that component. I use bounded queues, connection pooling, rate limits and backpressure to prevent cascading overload.”

15. Kafka vs synchronous REST

Characteristic	REST	Kafka
Communication	Request-response	Event streaming
Caller waits	Usually waits for result	Producer need not wait for consumer processing
Coupling	Temporal dependency	More decoupled
Replay	Separate implementation needed	Supported through retained records and offsets
Failure handling	Timeouts, retries, circuit breakers	Retries, offsets, error handling and dead-letter patterns
Good fit	Immediate query or decision	Durable events and asynchronous workflows

Use REST when the caller needs an immediate result, such as checking inventory. Use Kafka when a confirmed order should trigger independent notification, analytics and shipping consumers.

Exactly-once caution: Kafka transactions can support exactly-once processing within supported Kafka workflows, but updating an external database and committing a Kafka offset are separate operations unless coordinated by an appropriate design. Use idempotent consumers, unique event IDs, database constraints, and inbox/outbox patterns where appropriate.

Interview answer: “I choose REST for immediate request-response needs and Kafka for durable asynchronous events, replay and decoupled consumers. In both cases, I explicitly design for failure, duplicates and observability.”

Round 4 — HR and Managerial

16. Tell me about a challenging production issue

Use STAR: **Situation** (context and impact), **Task** (your responsibility), **Action** (investigation and fix), and **Result** (measured outcome and prevention).

Illustrative sample — adapt to a real incident:

“During production support, a payment-related API experienced a significant increase in latency. Requests that normally completed quickly were taking several seconds, and some timed out. My responsibility was to identify the root cause and restore normal performance without compromising payment correctness.

“I checked monitoring dashboards, application logs and distributed traces. The evidence showed that database queries were taking longer than usual. I reviewed the execution plan and identified an inefficient query scanning a large number of records. I worked with the database team to optimize the query and introduce an appropriate index after evaluating its impact. We tested the change, deployed it through the release process, and monitored latency and error rates.”

“After recovery, we added monitoring for query latency and documented the troubleshooting steps. This helped us detect similar issues earlier.”

Follow-up questions

- How did you identify the root cause rather than the symptom?
- How did you decide whether to roll back or hotfix?
- What was your rollback plan?
- How did you avoid duplicate payments during retries?
- How did you communicate with business and support teams?
- What measurable improvement did you achieve?
- What preventive action did you take?

Do not invent metrics or present an illustrative incident as your own. Explain the evidence, your contribution, verification and prevention.

17. Why JP Morgan Chase?

Sample answer:

“I am interested in JP Morgan Chase because financial technology requires a strong combination of engineering quality, security, reliability and scalability. My experience with Java, Spring Boot, REST APIs, database transactions and microservices has helped me understand how to build maintainable backend systems.

“I am particularly interested in challenges involving high-volume transactions, distributed systems, fault tolerance and data consistency. I would like to contribute my backend development experience while learning from engineers who build and operate systems at a large scale. I also value an engineering environment where code quality, testing, security and operational reliability are important.”

Possible follow-ups

- Why do you want to leave your current company? Focus on growth and role alignment, not criticism.
- Why should we hire you? Connect your skills to the position and give evidence.
- What do you know about our technology challenges? Discuss general financial-system needs unless you have verified team-specific information.

18. Handling technical disagreements

Sample answer:

“In a technical disagreement, I first try to understand the reasoning behind the other approach. I bring the discussion back to measurable requirements such as response time, failure handling, data consistency and maintainability.

“For example, if we disagree about Kafka versus REST, I compare whether the caller needs an immediate response, whether consumers need replay, and how each option handles downstream failures. If needed, I create a small proof of concept or review the alternatives with the team. Once we reach a decision, I document the trade-offs and help implement it. My focus is selecting the best solution for the business, not proving that my initial opinion was correct.”

Raise serious security, financial-correctness or data-loss risks explicitly and follow the organization's escalation process.

19. Long-term career goals

Sample answer:

“Over the next few years, I want to deepen my expertise in Java, Spring Boot, microservices and distributed systems. I want to become more effective at designing reliable services, investigating complex production issues and making sound architectural decisions.

“I also want to improve my understanding of cloud infrastructure, observability, performance engineering and secure software development. As I grow, I would like to take greater ownership of technical design, mentor junior developers, contribute to code reviews and collaborate closely with product and architecture teams. My goal is to become a strong senior engineer who can deliver business value while maintaining high standards of quality and reliability.”

Follow-up questions

- Do you want to become a technical lead or architect?
- How are you improving your skills?
- How do you mentor junior engineers?
- How do you balance delivery deadlines with code quality?

Final revision checklist

Round 1 — Core Java and DSA

- ■ Explain HashMap collisions, resizing and treeification
- ■ Solve longest substring without repeating characters
- ■ Detect a linked-list cycle and find its entry
- ■ Explain race conditions, synchronization and volatile
- ■ Implement equals() and hashCode() correctly

Round 2 — Spring Boot and Microservices

- ■ Explain @Transactional proxies, rollback and propagation
- ■ Explain self-invocation and how to fix it
- ■ Design an idempotent REST API
- ■ Use timeouts, retries, circuit breakers and bulkheads
- ■ Troubleshoot a slow API using metrics and traces

Round 3 — System Design

- ■ Design a highly available payment system
- ■ Explain saga, eventual consistency and transactional outbox
- ■ Diagnose memory leaks and thread pool exhaustion
- ■ Scale an API while protecting downstream resources
- ■ Choose between Kafka and synchronous REST

Round 4 — HR and Managerial

- ■ Explain a real production incident using STAR
- ■ Prepare a specific and credible Why JP Morgan answer
- ■ Describe a technical disagreement and its resolution
- ■ Explain realistic long-term career goals

Recommended preparation order

- HashMap internals and sliding-window DSA.
- @Transactional, proxy behavior and self-invocation.
- Idempotency and downstream failure handling.
- Payment-system design and microservices consistency.
- Production troubleshooting using real evidence and metrics.

Practice method: For each topic, speak for 2–3 minutes without reading. Then write the relevant code from memory and answer at least three follow-up questions. Use real project examples and be precise about what you personally implemented.

Technical accuracy note: Framework behavior can depend on Spring, Java, database and Resilience4j versions. Verify exact configuration and implementation details against the versions used in your project.